# Scrapy — Developer Guide

> Covers the repository at version **2.19.0** (`scrapy/VERSION`), Python **3.11+**, BSD-3-Clause. Official docs: https://docs.scrapy.org/ · Source of truth for narrative docs in this repo: `docs/` (Sphinx, reStructuredText).

Scrapy is a high-level web crawling and scraping framework. You write **spiders** (classes that say which URLs to fetch and how to parse the responses); Scrapy handles scheduling, concurrency, retries, redirects, cookies, throttling, deduplication, and exporting the scraped items.

This guide has two halves:

1. **Using Scrapy** — install, create a project, write spiders, run and export.
2. **Working on Scrapy** — repository layout, architecture, tests, and contribution rules.

---

## Part 1 — Using Scrapy

### 1.1 Install

```
pip install scrapy                    # from PyPI
pip install -e .                      # from this checkout (hatchling build backend)
pip install "scrapy[images,s3]"       # with optional extras
```

Optional extras (from `pyproject.toml`): `bpython`, `clipboard`, `color`, `gcs`, `httpx`, `images` (Pillow), `ipython`, `memusage` (psutil), `ptpython`, `robotparser`, `s3` (boto3/aioboto3), `socks`, `twisted-http2`, `uvloop`.

Core dependencies include Twisted, parsel (selectors), lxml, w3lib, itemadapter/itemloaders, protego (robots.txt), queuelib, aiohttp, cryptography/pyOpenSSL.

### 1.2 Create a project

```
scrapy startproject myproject
cd myproject
scrapy genspider example example.com      # templates: basic (default), crawl, csvfeed, xmlfeed
```

Generated layout (from `scrapy/templates/project/`):

```
myproject/
├── scrapy.cfg              # marks the project root; points at the settings module
└── myproject/
    ├── __init__.py
    ├── items.py            # item definitions (optional)
    ├── middlewares.py      # spider & downloader middleware skeletons
    ├── pipelines.py        # item pipeline skeleton
    ├── settings.py         # project settings
    └── spiders/            # your spiders live here
```

The generated `settings.py` sets polite defaults: `ROBOTSTXT_OBEY = True`, `CONCURRENT_REQUESTS_PER_DOMAIN = 1`, `DOWNLOAD_DELAY = 1`, `FEED_EXPORT_ENCODING = "utf-8"`.

### 1.3 Write a spider

```
import scrapy


class QuotesSpider(scrapy.Spider):
    name = "quotes"                           # unique id used by `scrapy crawl`
    allowed_domains = ["quotes.toscrape.com"]    # enforced by OffsiteMiddleware
    start_urls = ["https://quotes.toscrape.com/"]
    custom_settings = {"DOWNLOAD_DELAY": 0.5}    # per-spider overrides

    def parse(self, response):
        for quote in response.css("div.quote"):
            yield {
                "text": quote.css("span.text::text").get(),
                "author": quote.css("small.author::text").get(),
                "tags": quote.css("a.tag::text").getall(),
            }
        yield from response.follow_all(css="li.next a", callback=self.parse)
```

Key `scrapy.Spider` members (`scrapy/spiders/__init__.py`):

| Member | Purpose |
|---|---|
| `name` | Required, unique spider name. |
| `allowed_domains` | Requests to other domains are dropped by `OffsiteMiddleware`. |
| `start_urls` | URLs requested by the default `start()`. |
| `async def start(self)` | Yields initial requests/items. Override for custom start logic (replaces the older `start_requests()`). |
| `parse(self, response)` | Default callback. May be a regular generator or `async def`. |
| `custom_settings` / `update_settings(cls, settings)` | Per-spider settings. |
| `from_crawler(cls, crawler, ...)` | Factory with access to `crawler.settings`, `crawler.stats`, signals. |

| Member | Purpose |
|---|---|
| `self.logger`, `self.settings`, `self.crawler`, `self.state` | Runtime helpers. |
| `closed(self, reason)` | Called when the spider closes. |

Custom start:

```
async def start(self):
    for url in ["https://example.com/a", "https://example.com/b"]:
        yield scrapy.Request(url, callback=self.parse_page, cb_kwargs={"section": "a"})
```

Generic spiders in `scrapy/spiders/`: `CrawlSpider` + `Rule` (rule-based link following, `crawl.py`), `SitemapSpider` (`sitemap.py`), `XMLFeedSpider` / `CSVFeedSpider` (`feed.py`).

```
from scrapy.linkextractors import LinkExtractor
from scrapy.spiders import CrawlSpider, Rule

class ShopSpider(CrawlSpider):
    name = "shop"
    start_urls = ["https://example.com/"]
    rules = [
        Rule(LinkExtractor(allow=r"/category/")),                       # follow only
        Rule(LinkExtractor(allow=r"/product/"), callback="parse_item"), # parse
    ]
    def parse_item(self, response):
        yield {"url": response.url, "title": response.css("h1::text").get()}
```

### 1.4 Requests, responses and selectors

- `scrapy.Request(url, callback=..., errback=..., method="GET", headers=..., cookies=..., meta=..., cb_kwargs=..., dont_filter=False, priority=0)`
- `scrapy.FormRequest(url, formdata={...})`, `FormRequest.from_response(response, formdata=...)`, `scrapy.http.JsonRequest(url, data={...})`
- `response.follow(link_or_selector, callback)` / `response.follow_all(...)` — resolves relative URLs.
- Selectors (parsel): `response.css("a::attr(href)").getall()`, `response.xpath("//h1/text()").get()`, `.re(r"...")`, `response.json()` for JSON bodies.
- Useful request meta keys: `dont_redirect`, `dont_retry`, `handle_httpstatus_list`, `proxy`, `download_timeout`, `download_maxsize`, `max_retry_times`, `cookiejar`.

Non-2xx responses are filtered by `HttpErrorMiddleware` unless allowed via `handle_httpstatus_list` / `HTTPERROR_ALLOWED_CODES`.

### 1.5 Items, loaders, pipelines

Items can be plain dicts, `dataclasses`, `attrs` classes, or `scrapy.Item` subclasses — all handled through `itemadapter.ItemAdapter`.

```
class Product(scrapy.Item):
    name = scrapy.Field()
    price = scrapy.Field()
```

Item Loaders (`scrapy.loader.ItemLoader`) apply input/output processors to fill items.

Pipelines process every yielded item in order of their priority number (lower runs first):

```
# pipelines.py
from itemadapter import ItemAdapter
from scrapy.exceptions import DropItem

class PricePipeline:
    def process_item(self, item):
        adapter = ItemAdapter(item)
        if not adapter.get("price"):
            raise DropItem("missing price")
        return item

# settings.py
ITEM_PIPELINES = {"myproject.pipelines.PricePipeline": 300}
```

Built-in media pipelines: `scrapy.pipelines.files.FilesPipeline` and `scrapy.pipelines.images.ImagesPipeline` (needs Pillow).

### 1.6 Running and exporting

```
scrapy crawl quotes                         # run a project spider
scrapy crawl quotes -O quotes.json          # overwrite output file
scrapy crawl quotes -o quotes.jsonl         # append
scrapy crawl quotes -o out.csv:csv          # explicit format
scrapy crawl quotes -a category=books       # spider argument → self.category
scrapy crawl quotes -s LOG_LEVEL=INFO       # override a setting
scrapy runspider my_spider.py -O out.json   # single file, no project needed
```

Feed formats: `json`, `jsonlines`/`jsonl`, `csv`, `xml`, `pickle`, `marshal`. Storages: local file, FTP, S3, GCS, stdout. Configure persistently with the `FEEDS` setting:

```
FEEDS = {"data/%(name)s_%(time)s.jsonl": {"format": "jsonlines", "overwrite": True}}
```

Pause/resume a long crawl: `scrapy crawl big -s JOBDIR=crawls/big-1` (Ctrl-C once, re-run the same command to resume).

### 1.7 Command-line tool

All commands live in `scrapy/commands/`. Entry point: `scrapy = scrapy.cmdline:execute`.

| Command | Project? | Purpose |
|---|---|---|
| `startproject <name> [dir]` | no | Create a new project |
| `genspider [-t tmpl] <name> <domain>` | no | Generate a spider from a template |
| `crawl <spider>` | yes | Run a spider by name |
| `runspider <file.py>` | no | Run a spider from a file |
| `list` | yes | List spiders |
| `check <spider>` | yes | Run spider contracts |
| `parse <url>` | yes | Parse a URL with its spider and print results |
| `edit <spider>` | yes | Open a spider in `$EDITOR` |
| `shell [url\|file]` | no | Interactive scraping console |
| `fetch <url>` | no | Download a URL and print it |
| `view <url>` | no | Open a URL in the browser as Scrapy sees it |
| `genrequest [<curl>]` | no | Convert a curl command into `Request` code |
| `settings --get NAME` | no | Print a setting value |
| `bench` | no | Quick benchmark |
| `version [-v]` | no | Print version (and dependency versions) |
| `complete <bash\|fish\|zsh>` | no | Print a shell completion script |

Common global options: `-s NAME=VALUE`, `-a NAME=VALUE`, `-o/-O FILE[:FORMAT]`, `-L LEVEL`, `--nolog`, `--pdb`.

`scrapy shell "https://example.com"` gives you `response`, `request`, `fetch(url)`, `view(response)` — the fastest way to develop selectors.

### 1.8 Settings

Precedence (highest first): command line (`-s`) → spider `custom_settings` / `update_settings` → project `settings.py` → add-ons → defaults (`scrapy/settings/default_settings.py`).

Selected defaults:

| Setting | Default |
|---|---|
| `CONCURRENT_REQUESTS` | `16` |
| `CONCURRENT_REQUESTS_PER_DOMAIN` | `8` |
| `DOWNLOAD_DELAY` | `0` (project template sets `1`) |
| `DOWNLOAD_DELAY_JITTER` | `0.5` (replaces deprecated `RANDOMIZE_DOWNLOAD_DELAY`) |
| `DOWNLOAD_TIMEOUT` | `180` s |
| `DOWNLOAD_MAXSIZE` | 1 GiB |
| `ROBOTSTXT_OBEY` | `False` (project template sets `True`) |
| `COOKIES_ENABLED` | `True` |
| `RETRY_ENABLED` / `RETRY_TIMES` | `True` / `2` |
| `REDIRECT_MAX_TIMES` | `20` |
| `DEPTH_LIMIT` | `0` (unlimited) |
| `AUTOTHROTTLE_ENABLED` | `False` |
| `HTTPCACHE_ENABLED` | `False` |
| `LOG_LEVEL` | `"DEBUG"` |
| `CLOSESPIDER_ITEMCOUNT` / `_PAGECOUNT` / `_TIMEOUT` / `_ERRORCOUNT` | `0` (disabled) |
| `TWISTED_REACTOR` | `twisted.internet.asyncioreactor.AsyncioSelectorReactor` |
| `TWISTED_REACTOR_ENABLED` | `True` |
| `REMOTE_CONTROL_ENABLED` | `True` (extension used by the Scrapy MCP server) |
| `USER_AGENT` | `Scrapy/<version> (+https://scrapy.org)` |

Component dicts (`DOWNLOADER_MIDDLEWARES`, `SPIDER_MIDDLEWARES`, `EXTENSIONS`, `ITEM_PIPELINES`, `DOWNLOAD_HANDLERS`, `ADDONS`) are merged with their `*_BASE` counterparts; set a value to `None` to disable a built-in component.

### 1.9 Running from a script

```
from scrapy.crawler import AsyncCrawlerProcess
from scrapy.utils.project import get_project_settings

process = AsyncCrawlerProcess(get_project_settings())
process.crawl("quotes")          # spider name or class
process.start()                  # blocks until done
```

Other runners in `scrapy/crawler.py`: `CrawlerProcess`, `CrawlerRunner`, `AsyncCrawlerRunner` (when you manage the reactor/event loop yourself).

**Without Twisted reactor (experimental, since 2.15):** set `TWISTED_REACTOR_ENABLED = False`; Scrapy then runs on a plain asyncio loop and uses the aiohttp download handler for HTTP(S) by default (httpx handler available via `DOWNLOAD_HANDLERS`).

### 1.10 Debugging and operations tips

- `scrapy shell <url>` and `scrapy parse --spider=NAME -c parse_item <url>` to test callbacks.
- `from scrapy.shell import inspect_response; inspect_response(response, self)` inside a callback.
- `HTTPCACHE_ENABLED = True` while developing to avoid re-downloading.
- `AUTOTHROTTLE_ENABLED = True` for adaptive politeness.
- Stats are printed at the end of each crawl; access in code via `self.crawler.stats`.
- Telnet console and the `RemoteControl` extension expose a running crawl (the latter is used by the Scrapy MCP server; see `docs/topics/agents.rst`).

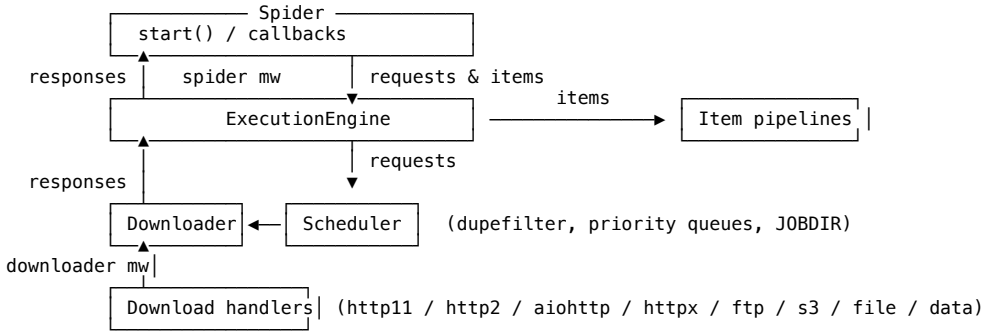
---

## Part 2 — Working on Scrapy

### 2.1 Repository layout

```
scrapy/                     the package
├── cmdline.py              CLI entry point (`scrapy` command)
├── commands/               one module per CLI command
├── crawler.py              Crawler, CrawlerProcess/Runner, Async* variants
├── core/
│   ├── engine.py           ExecutionEngine: drives the crawl loop
│   ├── scheduler.py        request queueing + dupe filtering
│   ├── scraper.py          runs callbacks, sends items to pipelines
│   ├── spidermw.py         spider middleware manager
│   ├── downloader/         Downloader, slots, middleware manager, handlers (http11, http2, aiohttp, httpx, ftp/aioftp, s3, file, data)
│   └── _http2/             HTTP/2 protocol implementation
├── downloadermiddlewares/ offsite, robotstxt, retry, redirect, cookies, httpcache, httpcompression, httpproxy, useragent, …
├── spidermiddlewares/      start, httperror, referer, urllength, depth, …
├── extensions/             corestats, logstats, closespider, feedexport, throttle, telnet, memusage, remote_control, …
├── pipelines/              media/files/images pipelines
├── spiders/                Spider, CrawlSpider, SitemapSpider, feed spiders
├── http/                   Request/Response classes, cookies, headers
├── selector/               Selector (wraps parsel)
├── linkextractors/         LinkExtractor
├── loader/                 ItemLoader
├── settings/               Settings classes + default_settings.py
├── contracts/              spider contracts (`scrapy check`)
├── templates/              startproject / genspider templates
├── utils/                  shared helpers
├── exporters.py            feed item exporters
├── signals.py, signalmanager.py, statscollectors.py, addons.py, …
docs/                       Sphinx docs (topics/, intro/, news.rst)
tests/                      pytest suite (+ mockserver/, sample_data/)
tests_typing/               typing tests
extras/                     maintainer scripts (benchmarks, coverage, release helpers, man page)
sep/                        Scrapy Enhancement Proposals (historical)
tox.ini, pyproject.toml, conftest.py, codecov.yml
```

### 2.2 Architecture and data flow




1. Engine gets initial requests from the spider's `start()` (through spider middlewares).
2. Requests go to the Scheduler, which deduplicates (`dupefilters.py`) and prioritises (`pqueues.py`, `squeues.py`).
3. Engine pulls requests and sends them to the Downloader through downloader middlewares (`process_request`).
4. Download handlers fetch; the response travels back through downloader middlewares (`process_response`).
5. Scraper passes the response through spider middlewares to the callback.
6. Callbacks yield items (→ pipelines) and new requests (→ scheduler).
7. Loop ends when nothing is pending; `spider_closed` fires, stats are dumped.

Cross-cutting: **signals** (`scrapy/signals.py`) for event hooks, **stats** (`statscollectors.py`), **extensions** (enabled via `EXTENSIONS`), **add-ons** (`ADDONS`, `addons.py`) which can modify settings and register components. Components are built with `build_from_crawler()` (`scrapy/utils/misc.py`) and usually expose `from_crawler(cls, crawler)`.

Scrapy runs on Twisted by default with the asyncio reactor, so `async def` callbacks, middlewares and pipelines are supported; the reactorless mode runs on asyncio only.

### 2.3 Development setup

```
git clone https://github.com/scrapy/scrapy && cd scrapy
python -m venv .venv && source .venv/bin/activate
pip install -e . tox pre-commit
pre-commit install
```

### 2.4 Tests and checks

Tests use pytest and are run through tox (environments defined in `tox.ini`).

```
tox                                 # default env, full suite
tox -e py -- tests/test_crawl.py    # args after `--` go to pytest
tox -e py314 -- scrapy tests -n auto  # parallel with pytest-xdist
tox -e py313,py314 -p auto          # several envs in parallel
tox -e min -- tests/test_foo.py     # oldest supported dependency versions
tox -e extra-deps                   # with optional dependencies
tox -e no-reactor                   # reactorless mode
tox -e default-reactor              # non-asyncio Twisted reactor
```

Required checks before finishing a change (from `AGENTS.md`):

```
pre-commit run --all-files     # ruff etc.
tox -e mypy
tox -e pylint
tox -e docs && tox -e docs-tests   # if you touched docs/docstrings
```

Test conventions: cover every added line (`# pragma: no cover` for unreachable branches), test via public APIs, use `pytest.mark.parametrize`, build components with `build_from_crawler()`, test spiders against the mock server (`tests/mockserver/`), only use `example.com` / `.example` domains.

### 2.5 Code and docs conventions

- Smallest change that solves the issue; no speculative settings/abstractions.
- New private names start with _; don't re-export or document private APIs.
- No module docstrings; document classes/functions. Log messages are f-strings.
- New boolean parameters are keyword-only. `assert` for invariants only.
- Fully typed: `mypy --strict` (see `[tool.mypy]` in `pyproject.toml`).
- Docs: API reference comes from docstrings via autodoc; narrative docs in `docs/topics/`. Use `versionadded`/`versionchanged` with literal `VERSION`. Do **not** edit `docs/news.rst` (written at release time). Wrap at 79 chars, sentence-case titles.

### 2.6 Git / PR workflow

- One change per PR; squash-merged. PR description: one or two sentences + `Closes #N`.
- After review, only add commits (merge `master` to catch up) — never rebase/amend/force-push.
- See `CONTRIBUTING.md`, `docs/contributing.rst`, `SECURITY.md` (vulnerability reporting), `CODE_OF_CONDUCT.md`.

### 2.7 Where to look next

| Question | File |
|---|---|
| What does setting X default to? | `scrapy/settings/default_settings.py` |
| How is a crawl started/stopped? | `scrapy/crawler.py`, `scrapy/core/engine.py` |
| How is a request scheduled/deduplicated? | `scrapy/core/scheduler.py`, `scrapy/dupefilters.py` |
| Which middlewares run, in which order? | `*_BASE` dicts in `default_settings.py` |
| How does a command work? | `scrapy/commands/<name>.py` |
| Narrative docs for a topic | `docs/topics/<topic>.rst` |
| What changed recently? | `docs/news.rst` |